QCLAIRVOYANCE QUANTUM LABS
Rewriting Reality With Quantum Technology

Cryptographic inventory and the CBOM

Beyond code and the wire: identity, cloud, HSMs and binaries

Preet Raj Gupta

Cryptography Analyst, QClairvoyance Quantum Labs

ISEA-ISAP 2026 · Tutorial



Outline

Where is our cryptography, *and how do we know we have found all of it?*



Why inventory comes first
Mosca, evidence



What counts as an asset
the CBOM model



The three common views
briefly, one run



Beyond code, files and the wire
identity, cloud, HSMs, binaries

MAIN PART



From inventory to decision
Mosca per asset



Open problems
research gaps

No laptop needed.

THE TUTORIAL PATH

Why
the evidence

Assets
the CBOM model

Basics
code, files, wire

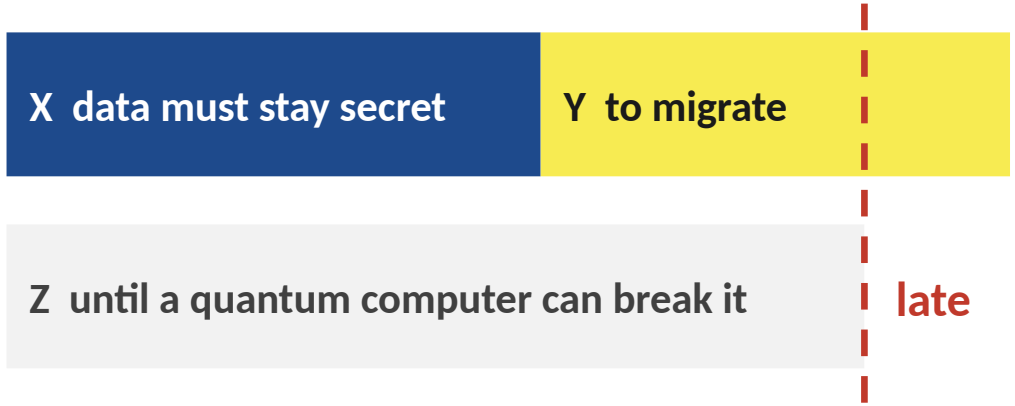
Beyond
identity, cloud, HSM, binary

Decide
Mosca per asset

Research
open problems

Inventory comes before migration

Mosca's inequality



If X + Y > Z, data recorded today is exposed before migration finishes. Y can only be estimated for cryptography that has been found.

What India's guidance asks for

DST / NQM Roadmap to Quantum Resiliency (May 2026)

Inventory is the first step of Milestone 1, and CBOMs from vendors are mandated from FY 2027-28.

CERT-In Technical Guidelines v2.0

Define the minimum elements a CBOM must carry (Tables 8 and 9).

Key exchange has moved; certificates have not

Dubey et al., IIT Jammu, 2026: top sites plus Indian bank and government domains

49%

support hybrid ML-KEM key exchange

0%

use post-quantum certificates

An inventory has to record key exchange and authentication separately, for every system.

Where inventories stall in practice

30%

of organisations have completed a cryptographic inventory

IBM IBV survey, 2025

~40%

of assets had no clear owner after a 14-month discovery programme

KU Leuven COSIC case study, 2026

What got in the way

Ownership Infrastructure, application team or vendor: nobody was sure.

Black boxes SaaS and payment platforms kept their cryptography behind contracts.

Legacy Old systems gave only partial or indirect evidence.

Operations Scanning had to fit production change windows.

IBM IBV, Secure the post-quantum future, Oct 2025, p. 12 · Zelenovic et al., arXiv 2605.16549, pp. 14-15

What counts as a cryptographic asset?

The four asset types in CycloneDX (ECMA-424), with BankLite examples

`algorithm`

A named primitive and its parameters
BankLite: ML-KEM-768, SHA-1, AES-128-ECB, HMAC-SHA256

`certificate`

Subject, issuer, validity and the algorithms it uses
BankLite: api.banklite.local, ECDSA P-256 with SHA-256

`protocol`

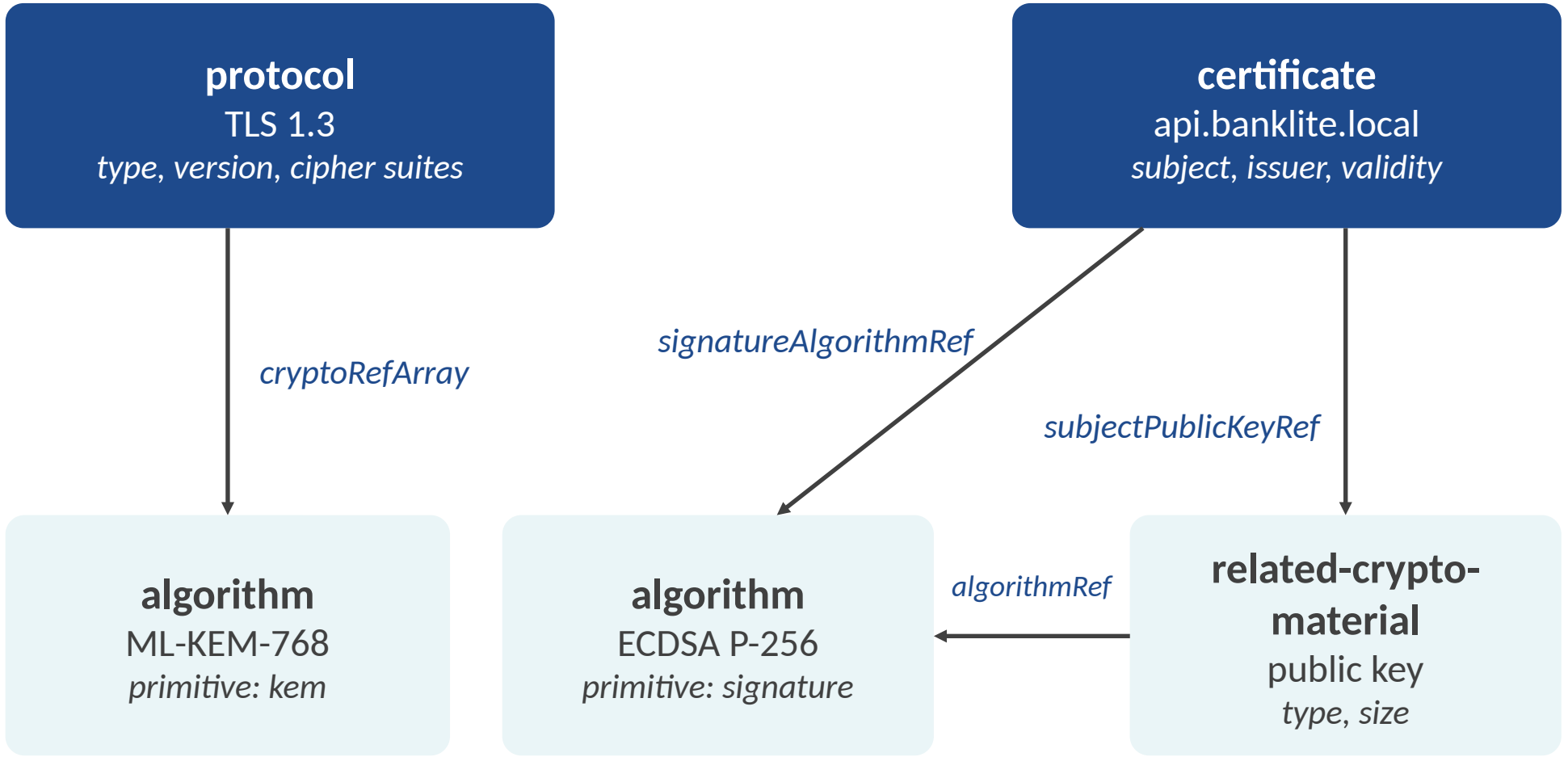
A protocol, its version and the cipher suites it offers
BankLite: TLS 1.2 on :8444 still accepting AES128-SHA

`related-crypto-material`

Keys, secrets, keystores and other material
BankLite: an unencrypted RSA-2048 private key backup

A CBOM is a graph

One BankLite TLS endpoint, as CycloneDX models it



A CBOM entry

CBOMKit-theia output for certs/api.crt, trimmed (CycloneDX 1.7)

```
{
  "type": "cryptographic-asset",
  "bom-ref": "2b47b9e4-…",
  "name": "api.banklite.local",
  "cryptoProperties": {
    "assetType": "certificate",
    "certificateProperties": {
      "subjectName": "api.banklite.local",
      "notValidAfter": "2029-01-14T03:53:32Z",
      "signatureAlgorithmRef": "5173ffa8-…",
      "subjectPublicKeyRef": "4a170e5e-…"
    } } }
```

`assetType`
Which of the four kinds this is.

`signatureAlgorithmRef`
Points to the ECDSA-SHA256 algorithm component.

`subjectPublicKeyRef`
Points to the certificate's key, which points to its algorithm.

These references make the CBOM a graph. A merge can break them.

THE TUTORIAL PATH

- **Why** — the evidence
- **Assets** — the CBOM model
- **Basics** — code, files, wire
- **Beyond** — identity, cloud, HSM, binary
- **Decide** — Mosca per asset
- **Research** — open problems

Three views of cryptography

Each answers a different question and has its own blind spot

Wire
live handshake

Answers
What is negotiated?

Cannot see
Anything that never crosses the network

Tool: openssl s_client, testssl.sh

Files
certificates, keys, config

Answers
What is deployed?

Cannot see
Whether it is ever used

Tool: CBOMKit-theia

Code
source code

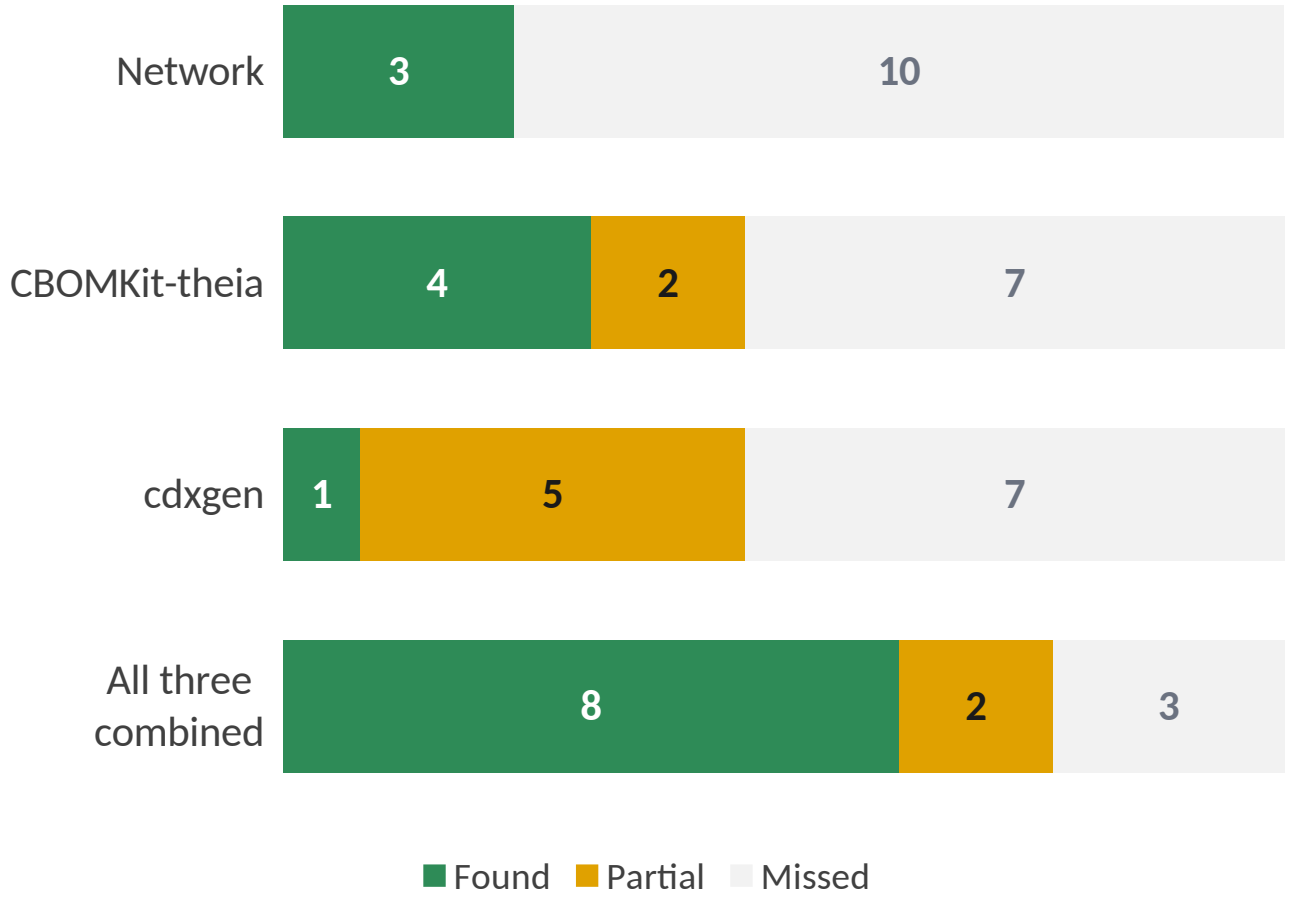
Answers
What did the developer intend?

Cannot see
Choices made in config or at runtime

Tool: cdxgen

Coverage by view

Assets found per view, out of 13



4 / 13

best single view

8 / 13

all three together

3

missed by every tool

Why three assets were missed

From a fixable bug to a fundamental limit

1 A tool bug

A07 AES-128-ECB in card.js

cdxgen detects the cipher, then drops it while normalising its name.

Fix: report it upstream

2 A missing view

A11 SHA256withRSA in SettlementSigner.java

No tool in the lab scans Java source code.

Fix: add a scanner for that language

3 A runtime choice

A13 AES-256-CBC in vault.js

The cipher name is assembled at runtime, from pieces.

No static tool can be sound and complete here in general (Rice's theorem)

easy to fix → fundamental

Measuring discovery needs ground truth

Real code has no answer key, so benchmarks plant known assets

Precision

correct findings / everything reported

How much of the CBOM can you trust?

Recall

real assets found / all real assets

How much of the system did you see?

Planted benchmarks

- Cryben (Näther and Hirsch, 2026): planted occurrences in source code.
- BankLite: 13 planted assets across network, files and code.

BankLite, all three views: recall 8 / 13, or 10 / 13 counting partial matches. A reported recall should say which.

Seven surfaces

Where cryptography can be observed, and what each observation cannot see

Surface	Reveals	Cannot see
Code	Crypto API calls	Runtime and configuration choices
Files	Certificates, keys, configuration	Whether they are ever used
Wire	Negotiated protocol and cipher suite	Anything not on the network
Identity	Identity-provider, token and SSH keys	Who relies on a token
Cloud	Customer keys, TLS policies	Provider-internal cryptography
HSM	Key objects and mechanisms	Which application uses a key
Binary	Compiled primitives	Parameters, keys and whether the code runs

Shaded rows: the main part of this tutorial. The first three were covered in part 3.

Identity and access

Keys that sign logins, tokens and sessions

Where it lives

- SAML signing certificates in the identity provider
- OpenID Connect key sets (JWKS) and JWT algorithms
- SSH host and user keys
- Kerberos encryption types

What it reveals

- Key type and size, signature algorithm, certificate expiry
- Which algorithms a token or ticket may use
- SSH key exchange and host-key algorithms offered

What it cannot see

- Which applications trust the tokens
- Whether a relying party still accepts a retired key
- Keys kept inside a hosted identity service

In the CBOM: a SAML certificate is a certificate; a JWKS key is related-crypto-material (public-key) with an algorithmRef (RS256: RSA with SHA-256); SSH is a protocol of type ssh.

```
# OpenID Connect: the keys the identity provider signs tokens with
curl -s https://idp.example/.well-known/openid-configuration | jq -r .jwks_uri
curl -s <jwks_uri> | jq '.keys[] | {kid, kty, alg, crv}'
ssh-audit host.example          # SSH key exchange, host keys, ciphers
klist -e                        # Kerberos encryption types
```

Cloud key management

Customer keys and managed TLS, listed through provider APIs

Where it lives

- Key-management services: AWS KMS, Azure Key Vault, Google Cloud KMS
- Certificate managers
- TLS policies on managed load balancers and API gateways

What it reveals

- Key spec, usage and protection level (software or HSM)
- Creation, rotation and expiry dates
- Which TLS policy each endpoint uses

What it cannot see

- Keys the provider manages for its own services
- Which workload uses which key, without the access logs
- Accounts, regions and projects that were not scanned

In the CBOM: a key is related-crypto-material with state, dates and securedBy (HSM or Software), linked to its algorithm; each TLS policy is a protocol with its cipher suites.

```
# AWS: list the keys, then the spec of each one
aws kms list-keys
aws kms describe-key --key-id <id> --query 'KeyMetadata.[KeySpec,KeyUsage,KeyManager]'
gcloud kms keys list --keyring <ring> --location <loc>    # algorithm, protection level
```

HSMs and key stores

Often the most important keys, and among the hardest to inventory

Where it lives

- General-purpose HSMs, reached through PKCS#11
- Key managers, reached through KMIP
- Payment HSMs, which use their vendor's own command set

What it reveals

- Key objects: class, type, size, label
- Attributes such as sensitive and extractable
- The mechanisms (algorithms) the HSM supports

What it cannot see

- The key material itself, by design
- Which application uses a key
- Payment keys stored outside the HSM, encrypted under its master key

In the CBOM: a key is related-crypto-material with size, id and securedBy (mechanism: HSM); its algorithm can carry executionEnvironment: hardware.

```
# PKCS#11 with OpenSC's pkcs11-tool (SoftHSM works for practice)
pkcs11-tool --module /usr/lib/softhsm/libsofthsm2.so --login --list-objects
pkcs11-tool --module /usr/lib/softhsm/libsofthsm2.so --list-mechanisms
```

Binaries and firmware

When there is no source code: appliances, terminals, vendor software

Where it lives

- Vendor appliances and closed-source software
- Firmware of payment terminals, routers and meters
- Statically linked and stripped programs

What it reveals

- Primitives, from constants such as S-boxes and NTT tables
- Instruction patterns and data-flow shapes
- Library names and versions, if not stripped

What it cannot see

- Parameters and modes, often
- Whether the code is ever called
- Which keys it uses

In the CBOM: a finding is an algorithm with implementationPlatform (for example armv8-a), and an evidence occurrence with the binary's path and offset.

```
binwalk -e firmware.bin                  # unpack a firmware image
capa ./service                           # capabilities, e.g. encrypt data using AES
yara crypto_signatures.yar ./service     # known constants, e.g. the AES S-box
```

Binaries: what research shows

From finding old cryptography to confirming that new cryptography shipped

Find proprietary primitives

Data-flow-graph matching with symbolic execution identifies and classifies primitives, including unknown proprietary ones.
Meijer et al., USENIX Security 2021

Compare the tools

Re-running existing binary tools showed there was no common benchmark, and gaps on modern primitives.
Fan et al., ACSAC 2024

Confirm PQC shipped

NTT constant tables identify ML-KEM and ML-DSA in stripped binaries; 12 programs with uncatalogued ML-KEM among 6,224 on one Linux system.
Junaid, 2026 preprint

Binary analysis can verify a migration: it checks what was compiled in, not what a vendor states.

One key, four places

The same RSA signing key, as each surface reports it

HSM	Files	Wire	Code
A private-key object Calls it: label and CKA_ID	**The certificate for its public key** Calls it: file path and serial number	**The same certificate in a handshake** Calls it: host, port and certificate	**A call to sign with SHA256withRSA** Calls it: source file and line

Four entries, one key. A public-key fingerprint links the asymmetric cases. Symmetric keys have no public half to match on.

Identity and cloud add more names for the same key: the service account allowed to use it, and the cloud key ID.

Into the CBOM

The same four asset types cover every surface

Surface	Finding	CycloneDX asset type and fields
Identity	SAML signing certificate	`certificate: signatureAlgorithmRef, subjectPublicKeyRef`
Identity	JWKS key, alg RS256	`related-crypto-material: public-key, size, algorithmRef`
Identity	SSH server	`protocol: type ssh, cipherSuites`
Cloud	KMS key, RSA_2048, HSM-backed	`related-crypto-material: state, dates, securedBy`
Cloud	Load-balancer TLS policy	`protocol: type tls, version, cipherSuites`
HSM	PKCS#11 private-key object	`related-crypto-material: private-key, size, id, securedBy HSM`
Binary	AES constants in firmware	`algorithm: implementationPlatform; evidence: location, offset`

Scoring each asset

Mosca ratio (X + Y) / Z with Z = 10 years, times exposure; reference worksheet

Asset	What	Quantum breaks it?	X years	Y years	Mosca ratio	Exposure	Priority
A03	TLS 1.2 with AES128-SHA	Y	10	3	1.3	2	**2.6**
A10	RSA settlement keystore	Y	8	3	1.1	2	**2.2**
A11	SHA256withRSA signing	Y	8	3	1.1	2	**2.2**
A12	OpenSSL policy, SECLEVEL=1	Y	10	1	1.1	2	**2.2**
A04	ECDSA P-256 certificate	Y	0	2	0.2	3	**0.6**
A09	HMAC-SHA256 webhooks	N	0	0	0.0	3	**0.0**

Long-lived signatures rank high. A10 and A11 protect no secrets, but settlement files must stay verifiable for years.

Not everything needs migration

The 13 BankLite assets in three groups (A06 belongs in two)

Quantum-vulnerable

A03 TLS 1.2 static RSA key transport
A04 ECDSA P-256 certificate
A05 RSA-2048 certificate
A06 RSA key backup
A10 RSA settlement keystore
A11 SHA256withRSA signing
A12 policy that re-enables A03

Classically broken today

A07 AES-ECB, hard-coded key
A08 SHA-1 receipt IDs
A13 zero IV, embedded key
A06 unencrypted key on disk

Fine as it is

A02 ML-KEM-768 (the target)
A09 HMAC-SHA256
A01 hybrid TLS 1.3 key exchange

Migrate first: A03, then A10 and A11 together, and A12. Fixing A12, one line of OpenSSL policy, also removes A03 on every service on the host.

Open problems

1

Ground truth beyond code

Benchmarks exist for source code and, partly, for binaries. There is none for identity, cloud or HSM inventories.

2

Linking across surfaces

Recognising one key in the HSM, on the certificate, on the wire and in the code, so that a merged CBOM lists it once.

3

Runtime and hybrids

Algorithms chosen at runtime need dynamic analysis; the classical half of a hybrid should not be flagged for migration.

4

Claims of completeness

What coverage an inventory can honestly report, and how to keep it current as systems change.

After the session

- Inventory every surface, not only the ones your tools scan.
- Validate every CBOM, and report its coverage with its findings.

Read next

Dubey and Varshney, PQ readiness of the Internet, arXiv 2606.16473 (2026)
Näther and Hirsch, Hidden Ciphers, arXiv 2608.04857 (2026)
Bögli, Spieler and Kehrer, BF-CBOM, ICPC 2026
Yamamuro, Uemura and Fukushima, SoK on discovery tools, ICISSP 2026
Zelenovic et al., PQ discovery as a governance capability, arXiv 2605.16549 (2026)
Meijer, Moonsamy and Wetzels, Where's Crypto?, USENIX Security 2021
Fan, Biswas and Garman, R+R: crypto function identification in binaries, ACSAC 2024
Junaid, PQC detection in stripped binaries, arXiv 2608.25122 (2026)

github.com/preet-qclair/banklite-lab

QCLAIRVOYANCE QUANTUM LABS
Rewriting Reality With Quantum Technology